Rust KV and Scheduling Core Project Primer

Purpose

The project builds a CPU-side key-value (KV) cache, scheduling, and serving-runtime core for large language model (LLM) inference in Rust. It serves three goals:

1. **Rust Depth:** Learn Rust in depth by building a nontrivial system.
2. **Dynamo Depth:** Learn NVIDIA Dynamo in depth and reach an upstream contribution, in support of the Dynamo Distinguished Engineer posting (JR2009883).
3. **Interview Preparation:** Prepare for the Photon Intelligence coding interview, conducted in Rust or Go with AI coding tools allowed.

Each milestone follows one pattern: Build the component, read Dynamo's counterpart, and compare the two. The build develops Rust skill. The comparison develops Dynamo knowledge and surfaces contribution candidates. The timeboxed core of each milestone serves as an interview drill.

Dynamo already ships Rust implementations of the router, the indexer, the KV Block Manager (KVBM), and the distributed runtime. The project is learning and evidence work, not a market gap.

Alignment With the Dynamo Posting

Posting Item	Project Coverage
Disaggregated serving	Milestone 7, now core
KV-aware routing	Milestones 3 and 5
Multimodal inference (image, video, audio)	Milestone 1 extension
Resiliency	Milestones 8 and 9
Performance and bottleneck analysis	Performance track, all milestones
Scalability	Milestone 3 concurrency, Milestone 8
Observability and operations	Observability track, all milestones
Storage partner teams	Milestone 10
Open-source contribution	Contribution lead
Rust, Python, and Go	Rust build; Go as interview fallback

Background: The KV Cache

- To generate each token, the attention layers require a Key vector and a Value vector for every earlier token. Retaining those vectors forms the KV cache.
- The KV cache resides in GPU memory in fixed-size blocks, for example 16 tokens per block, comparable to operating-system memory pages.
- Requests often share opening text, such as a system prompt. When those blocks are already cached, the engine skips recomputing them (prefill), which reduces Time to First Token (TTFT).
- The milestones cover finding, holding, scheduling, routing, isolating, moving, and recovering reusable blocks.

Milestone Map

#	Component	Dynamo Counterpart	Interview Drill
1	Block hashing	`lib/tokens`, `lib/kv-hashing`	Chained hasher, prefix match
2	Block pool	`lib/kvbm-logical`	Ref-counted allocator, eviction
3	Radix index	`lib/kv-router/src/indexer`	Insert, match, evict
4	Scheduler	`lib/mocker/src/scheduler`	Batching step under a token budget
5	KV router	`lib/kv-router/src/scheduling`	Overlap versus load scoring
6	Tenant keys	`lib/kv-hashing/src/salt.rs`	Keyed salt, fail closed
7	Disaggregated handoff	`lib/kv-router/src/conditional_disagg.rs`	Block handoff protocol
8	Discovery and events	`lib/runtime/src/discovery`, `lib/runtime/src/transports`	Lease registry with heartbeat
9	Request migration	`lib/llm/src/migration.rs`	Idempotent retry
10	Offload tier (optional)	`lib/kvbm-physical`	Two-tier cache

Paths are verified against the Dynamo `main` branch at commit `6e4221b` (October 2026).

Cross-Cutting Tracks

Observability Track

Every milestone emits per-request metrics and traces from its first build: TTFT, Time Per Output Token (TPOT), cache hit rate, queue depth, and block utilization. Traces follow one request across router, prefill, and decode once Milestone 7 exists.

- **Rust Concepts:** The `tracing` crate, spans, and metrics exporters.
- **Dynamo Reading:** `lib/llm/src/request_trace`, including the OpenTelemetry sink in `otel_sink.rs`.

Performance Track

Every milestone ends with a benchmark and a written bottleneck finding: What limits throughput or latency, the evidence, and the fix or the reason it was deferred.

- **Rust Concepts:** Criterion benchmarks, flamegraph profiling, allocation counting, and lock contention analysis.
- **Output:** A short findings log per milestone, which also serves as interview material on analyzing complex systems.

Milestone Detail

Milestone 1: Block Hashing

- **Problem:** The engine needs a low-cost test of whether a prompt's opening blocks are already cached.
- **Build:** Split the tokens into blocks and hash each block, mixing in the previous block's hash, so that each hash identifies the full prefix through that block.
- **Example:** Two prompts share their first 32 tokens and then differ. With 16-token blocks, their first two hashes match and the third differs, so the prefix match is two blocks.
- **Multimodal Extension:** Fold image, video, or audio content identifiers into the hash of the block where they occur. Requests carrying the same media at the same position can still share the blocks before it. Dynamo documents this design in `salt.rs`.
- **Rust Concepts:** Ownership, slices, traits, the `Hash` trait, and property-based tests.
- **Dynamo Reading:** `blocks.rs` and `radix.rs` in `lib/tokens`, then `lib/kv-hashing`, which defines the block identity contract shared by the router, KVBM, and the engines.
- **Proving Test:** Hashing a sequence in one pass yields the same hashes as hashing it incrementally while generation appends tokens.

Milestone 2: Block Pool

- **Problem:** GPU memory holds a fixed number of blocks, which must be allocated and reclaimed.
- **Build:** An allocator that keeps reference counts on shared prefix blocks and returns a block to the free pool only when its count reaches zero. Finished blocks remain cached for reuse until memory pressure triggers least recently used (LRU) eviction. Resource Acquisition Is Initialization (RAII) handles return blocks automatically through the `Drop` trait.
- **Rust Concepts:** `Drop`, `Arc` and `Weak`, interior mutability, and lifetimes.
- **Dynamo Reading:** `lib/kvbm-logical`, specifically `pools`, `blocks`, `manager`, and `tinylfu.rs`.
- **Proving Test:** No block is leaked, and no block is freed twice.

Milestone 3: Radix Index

- **Problem:** Comparing a prompt against every cached sequence does not scale.
- **Build:** A tree in which each root-to-node path is a sequence of block hashes and each node is a cached block. A lookup descends while hashes match. Radix compression merges chains of single-child nodes into one node. The operations are insert, longest-prefix match, and eviction of least recently used leaves. The index is built single-threaded first, then made concurrent.
- **Rust Concepts:** `Send` and `Sync`, locking strategy, and benchmarking with criterion.
- **Dynamo Reading:** `lib/kv-router/src/indexer`, including `concurrent_radix_tree_compressed`. The router uses this index to track which worker holds which blocks.
- **Proving Test:** Random operation sequences applied to the tree and to a brute-force list return identical results.

Milestone 4: Scheduler

- **Problem:** The GPU executes in steps. Each step decodes one token per running request and processes new prompts. A scheduler selects the contents of each step.
- **Build:**
  - **Continuous Batching:** Finished requests leave and waiting requests join after every step, rather than the batch waiting for its slowest request.
  - **Token Budget and Chunked Prefill:** Each step has a token cap. Long prompts are split across several steps so that requests in decode are not stalled.
  - **Preemption:** When KV memory is exhausted, a running request pauses. Its blocks are either copied to CPU memory (swap) or discarded and recomputed later (recompute).
  - **Targets:** TTFT and TPOT.

- **Rust Concepts:** Async with tokio, channels, and state machines.
- **Dynamo Reading:** `lib/mocker/src/scheduler`, Dynamo's model of engine scheduling.
- **Proving Test:** No step exceeds the token budget, and no request waits indefinitely.

Milestone 5: KV Router

- **Problem:** Several engine instances (workers) serve traffic, and each new request needs a worker.
- **Build:** A scoring function that weighs cached-prefix overlap against current load. The worker holding the most of the prompt saves the most prefill but may already be overloaded.
- **Rust Concepts:** Trait objects versus generics, and plugin design.
- **Dynamo Reading:** `lib/kv-router/src/scheduling`.
- **Proving Test:** Equal scores resolve to the same worker every time, for example the lowest worker ID.

Milestone 6: Tenant Keys

- **Problem:** Shared prefix caching leaks information. A faster first token reveals that another tenant sent the same prefix.
- **Build:** A per-tenant salt mixed into the first block's hash. Because hashes chain, every later hash changes, and tenants never share blocks. The salt uses a keyed hash, such as a Hash-based Message Authentication Code (HMAC) with a per-tenant secret derived from verified workload identity. A request without verified identity is rejected rather than served from the shared cache (fail closed).
- **Rust Concepts:** Cryptography crates, error types, and API design.
- **Dynamo Reading:** `lib/kv-hashing/src/salt.rs`.
- **Proving Test:** Two tenants sending the same prompt receive zero cache hits on each other's blocks.

Milestone 7: Disaggregated Handoff

- **Problem:** Prefill is compute-bound and decode is memory-bandwidth-bound. Dynamo can run them on separate workers and move KV blocks between them over the NVIDIA Inference Xfer Library (NIXL).
- **Build:** A prefill worker pool and a decode worker pool. A request is routed to prefill, its KV blocks are handed to a decode worker over a mock transport with configurable bandwidth and latency, and decode continues from those blocks. A conditional rule decides whether a request is worth disaggregating or should run on one worker.
- **Rust Concepts:** Zero-copy buffers, async transport traits, and backpressure.
- **Dynamo Reading:** `lib/kv-router/src/conditional_disagg.rs` for the routing decision, and `lib/kvbm-physical/src/transfer` for block transfer.
- **Proving Test:** A disaggregated run produces the same output tokens as an aggregated run of the same request.

Milestone 8: Discovery and Event Plane

- **Problem:** Workers join, leave, and fail. The router must know which workers exist and which blocks each one holds.
- **Build:** Workers register with a lease and renew it by heartbeat; an expired lease removes the worker. Workers publish block-stored and block-removed events, and the router consumes them to keep the Milestone 3 index current.
- **Rust Concepts:** Async streams, cancellation, and backpressure.
- **Dynamo Reading:** `lib/runtime/src/discovery` and `lib/runtime/src/transports`.
- **Proving Test:** After a worker's lease expires or it reconnects, the router index matches the actual worker state, with no events lost or applied twice.

Milestone 9: Request Migration

- **Problem:** A worker can fail mid-generation. The request should complete on another worker without the client noticing.
- **Build:** A retry manager that detects a failed stream, selects a new worker, replays the tokens already delivered, and resumes generation. Fault injection kills workers at controlled points.
- **Rust Concepts:** Error propagation, idempotency, and deterministic fault injection.
- **Dynamo Reading:** `lib/llm/src/migration.rs`. Dynamo replays delivered tokens and recomputes the KV cache on the new worker.
- **Proving Test:** A request whose worker is killed mid-generation completes on another worker with no duplicated or missing tokens.

Milestone 10 (Optional): Offload Tier

- **Problem:** GPU memory is small relative to reusable KV. Evicted blocks can move to CPU memory or disk instead of being discarded.
- **Build:** A second, larger, slower tier behind the Milestone 2 pool. Eviction demotes blocks, and a cache hit on the lower tier promotes them back.
- **Rust Concepts:** Async I/O, memory-mapped files, and trait-based tier abstraction.
- **Dynamo Reading:** `lib/kvbm-physical`, including `layout`, `manager`, and `transfer`.
- **Proving Test:** Under a working set larger than the first tier, hit rate exceeds the single-tier baseline.

Dynamo Coverage

Covered by Milestones 1 Through 9

- **Hashing Contract:** How the router, KVBM, and the engines agree on block identity, including multimodal content and salt.
- **KVBM Logical Layer:** Block lifecycle, reference counting, reuse, and eviction.
- **KV-Aware Routing:** The index and the scoring decision.
- **Engine Scheduling Model:** Batching, chunked prefill, and preemption as modeled in the mocker.
- **Disaggregated Serving:** The prefill and decode split and the block handoff.
- **Distributed Runtime:** Discovery, leases, and the event plane.
- **Resiliency:** Request migration under worker failure.
- **Multi-Tenancy Weakness:** The salt derivation in `salt.rs`.

Not Covered

- **KVBM Physical Tiers:** Unless Milestone 10 is completed.
- **Planner:** Resizing prefill and decode pools to meet latency targets. Implemented in Python.
- **Engine Connectors:** Integration of vLLM, SGLang, and TensorRT-LLM. Implemented in Python.
- **Frontend and Kubernetes Operator.**

Contribution Lead

`salt.rs` derives the salt by hashing the salt string with `xxh3_64_with_seed(value, 1)` and adding the result to the chain seed. xxh3 is a fast non-cryptographic hash, and the derivation uses no secret key. Anyone who knows or guesses the salt string can therefore reproduce the block hashes. Dynamo 1.3 documentation also describes the salt as taken from the unverified `x-tenant-id` header and as unsupported on SGLang.

- **Status:** Existing issues and pull requests have not been checked.
- **Path:** A keyed hash changes the hashing contract across the router, KVBM, and engine producers, so a design issue precedes code.
- **Lower-Risk First Pull Request:** Tests or documentation in a crate studied during Milestones 1 through 3. Milestones 8 and 9 add resiliency test coverage as a second candidate area, matching the posting's emphasis on resiliency.

Interview Preparation

Format

The interview is conducted in Rust or Go, with AI coding tools allowed. Weight therefore shifts from writing syntax toward decomposition, correctness, tests, and review of generated code.

Practice Protocol

- **Timebox:** Each drill runs 45 to 60 minutes, using the AI tool intended for the interview, with reasoning stated aloud.
- **Tests First:** Tests are written before generated code is accepted. They catch AI errors live and demonstrate judgment.
- **Rust Review:** Generated Rust often compiles only after added clones, `Arc<Mutex<_>>` wrappers, or lifetime workarounds that conceal a poor ownership design. Detecting and correcting these in review is the Rust depth the project builds.

Language Choice

- **Rust:** Matches Photon's runtime. Borrow-checker and concurrency errors cost time under a timebox.
- **Go:** The stronger production language from prior work and the lower-risk choice for speed.
- **Decision Rule:** Rust if Milestones 1 through 3 are complete before the interview. Otherwise Go, with the Rust project presented as evidence of direction.

The project remains in Rust regardless, because Goals 1 and 2 depend on it.

Environment

- **Milestones 1 Through 10:** CPU-only, on a MacBook Pro M1 Max (Jeeves7).
- **Real Forward Pass:** A later extension that calls FlashInfer from Rust through the foreign function interface (FFI) would run on a DGX Spark. That effort is not scoped.

Sources

- NVIDIA Dynamo Distinguished Engineer Posting (JR2009883): LinkedIn job listing, provided as a PDF
- NVIDIA Dynamo Repository: https://github.com/ai-dynamo/dynamo
- Dynamo `kv-hashing` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/kv-hashing
- Dynamo `kv-router` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/kv-router
- Dynamo `kvbm-logical` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/kvbm-logical
- Dynamo `kvbm-physical` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/kvbm-physical
- Dynamo `mocker` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/mocker
- Dynamo `runtime` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/runtime
- Dynamo `tokens` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/tokens
- Dynamo `llm` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/llm
- Dynamo KVBM Design: https://docs.nvidia.com/dynamo/design-docs/kvbm-design
- Dynamo Mocker Guide: https://docs.nvidia.com/dynamo/user-guides/testing/mocker
- Dynamo Request Migration: https://docs.nvidia.com/dynamo/dev/kubernetes/fault-tolerance/request-migration
- Dynamo 1.3 Request Extensions and Cache Salt: https://docs.nvidia.com/dynamo/v1.3.0/additional-resources/nvidia-request-extensions-nvext

Acronyms

Acronym	Expansion
FFI	Foreign Function Interface
HMAC	Hash-based Message Authentication Code
KV	Key-Value
KVBM	KV Block Manager
LLM	Large Language Model
LRU	Least Recently Used
NIXL	NVIDIA Inference Xfer Library
RAII	Resource Acquisition Is Initialization
TPOT	Time Per Output Token
TTFT	Time to First Token